

Reporte de Agentic Loop

1. ANÁLISIS DE ARQUITECTURA

ANÁLISIS ARQUITECTÓNICO - Validador de Prompts y Tokens

PATRÓN ARQUITECTÓNICO: Single-File Dual-Layer Architecture

Dada la restricción crítica de máximo 2 archivos, se adopta una arquitectura ultra-condensada donde cada archivo representa una capa completa del sistema. No se aplican patrones de separación de responsabilidades tradicionales; en cambio, se prioriza la cohesión funcional dentro de cada archivo.

CAPAS DEL SISTEMA:

1. Capa de Presentación + Lógica Cliente (index.html): Monolito frontend que integra estructura semántica HTML5, estilos utilitarios via Tailwind CDN, y lógica de negocio cliente en JavaScript vanilla. El estado de la UI se gestiona directamente en el DOM sin frameworks reactivos, reduciendo dependencias a cero.
2. Capa de API + Lógica de Negocio (main.py): Microservicio FastAPI que expone un único endpoint REST. Incluye configuración CORS permisiva para localhost, lógica de estimación de tokens (palabras * 1.3), y simulación de latencia asíncrona no bloqueante con `asyncio.sleep()` con rango aleatorio 500ms-2000ms.

DECISIONES TECNOLÓGICAS:

- FastAPI sobre Flask/Django: soporte nativo `async/await`, validación automática con Pydantic, documentación OpenAPI auto-generada, rendimiento superior para I/O bound.
- Tailwind CSS via CDN: elimina pipeline de build, clases utilitarias inline evitan CSS externo, consistencia visual inmediata.
- Fetch API nativa: sin dependencias externas, soporte `async/await`, manejo de errores con `try/catch`.
- `asyncio.sleep()` vs `time.sleep()`: garantiza no-bloqueo del event loop de uvicorn durante la simulación de latencia.

SEGURIDAD (DevSecOps):

- CORS configurado explícitamente solo para orígenes localhost (no wildcard en producción).
- Validación de payload en backend con Pydantic BaseModel (campo prompt requerido, tipo string).
- Validación en frontend antes de enviar (prevención de requests vacíos).
- Sin almacenamiento de datos sensibles, sin autenticación requerida (herramienta interna).
- Headers de seguridad básicos implícitos en FastAPI.
- Input sanitization implícita al tratar el prompt solo como string para conteo.

FLUJO DE DATOS:

Usuario escribe prompt ! Validación cliente (no vacío) ! POST /api/analyze con JSON body ! FastAPI valida schema Pydantic ! Cálculo tokens ($\text{len}(\text{words}) * 1.3$) ! `asyncio.sleep(\text{random } 0.5\text{-}2.0\text{s})` ! Response JSON {tokens, latency_ms, status} ! Frontend renderiza 3 tarjetas de resultado.

LÓGICA DE NEGOCIO - Estado del prompt:

- 'Óptimo': tokens estimados ≤ 1000
- 'Alerta': tokens estimados > 1000

ESTADOS DE UI:

1. Idle: textarea habilitado, botón activo 'Analizar Prompt'
2. Error vacío: textarea con borde rojo, sin request
3. Loading: botón deshabilitado con texto 'Analizando...'
4. Success: panel de 3 tarjetas visible con resultados

COMPATIBILIDAD: El index.html funciona abriéndose directamente en navegador (file://) gracias a la configuración CORS del backend que acepta origen null (equivalente a file://) además de localhost.

2. MÉTRICAS DE ITERACIÓN (QA)

- index.html | Estado: APROBADO | Iteraciones: 1
- main.py | Estado: APROBADO | Iteraciones: 1